Detecting Underspecification in Software Requirements via k-NN Coverage Geometry*

Wenyan Yang
Tampere University
wenyan.yang@tuni.fi

Tomáš Janovec
Tampere University
tomas.janovec@tuni.fi

Samantha Bavautdin
Tampere University
samantha.bavautdin@tuni.fi

Abstract

We propose GeoGap, a geometric method for detecting missing requirement categories in software specifications. The method represents each requirement as a unit vector via a pretrained sentence encoder, then measures coverage deficits through k-nearest-neighbour distances z-scored against per-project baselines. Three complementary scoring components—per-point geometric coverage, category-restricted distributional coverage, and annotation-free population counting—fuse into a unified gap score controlled by two hyperparameters. On the PROMISE NFR benchmark, GeoGap achieves 0.935 AUROC for detecting completely absent requirement categories in projects with $N \geq 50$ requirements, matching a ground-truth count oracle that requires human annotation. Six baselines confirm that each pipeline component—per-project normalisation, neural embeddings, and geometric scoring—contributes measurable value.

1 Introduction

Software requirements specifications are routinely incomplete. A healthcare system may specify authentication and audit logging but omit backup and disaster-recovery requirements that every peer system addresses. Detecting such *underspecification* before design commits resources prevents costly rework.

No universal checklist of required concerns exists: what a project *should* address depends on its domain, scale, and quality priorities. We therefore frame the problem as a **coverage comparison**: given a corpus of peer projects with known requirement categories, identify regions of concern the corpus covers but the target project does not.

We propose GeoGap, which represents each requirement as a unit vector on the hypersphere $\mathbb{S}^{d-1}$ via a pretrained sentence encoder, then measures coverage deficits via k-nearest-neighbour distances. The method has three scoring components:

- **Geometric coverage** (Ψ_{geo}): for each corpus point, is the target's nearest requirement unusually far?
- **Type-restricted coverage** (Ψ_{cat}): the same question, restricted to same-category requirements only.
- **Population count** (Ψ_{pop}): does the target have fewer requirements of each category than peer projects?

These fuse into a single score via two hyperparameters, then aggregate onto a human-readable (category $\times$ topic) grid. Figure 1 shows the pipeline.

*Project Page · Code · Overleaf

Technical Report (2026).

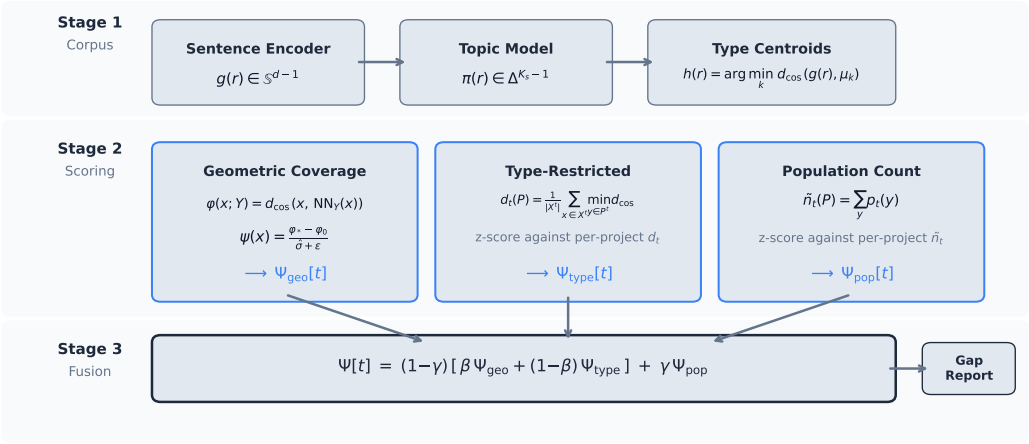


Figure 1: The GEOGAP pipeline. Stage 1 builds the reference corpus (embeddings, topics, category centroids). Stage 2 computes three complementary gap scores. Stage 3 fuses them and produces a structured gap report with heatmaps.

2 Problem Definition

2.1 Dataset

We use the PROMISE NFR dataset [2]: $N = 621$ natural-language requirement sentences from $M = 15$ software projects. Each requirement r has a project identifier $\mathrm{pid}(r) \in \{P_1, \ldots, P_{15}\}$ and a quality-attribute category from $K_c = 12$ classes (Availability, Fault Tolerance, Functional, Legal, Look & Feel, Maintainability, Operability, Performance, Portability, Scalability, Security, Usability). Project sizes range from 12 to 92; six have $N_j \geq 50$.

2.2 Gap Definition

Let P_* be a target project and $\mathcal{D} = \{P_1, \ldots, P_M\}$ a reference corpus. A sentence encoder g maps each requirement to a unit vector $g(r) \in \mathbb{S}^{d-1}$ ($d = 1024$).

Gap. A region of $\mathbb{S}^{d-1}$ where the corpus has substantial coverage (many points from multiple projects) but the target has little or no coverage.

We address three questions:

RQ1 Which types does the entire corpus cover but P_* does not? (Structural gaps)
RQ2 Which types do similar projects cover but P_* does not? (Domain-specific gaps)
RQ3 Which requirements in P_* have no analogue in the corpus? (Novelties)

3 Representation

Before defining the gap detection method, we establish the mathematical representation. Table 1 summarises the key symbols used throughout the paper.

3.1 Sentence Embedding

We encode every requirement sentence r into a d-dimensional unit vector using Qwen3-Embedding-0.6B [1]:

$$g(r) \in \mathbb{S}^{d-1}, \qquad \|g(r)\| = 1, \qquad d = 1024. \tag{1}$$

Here $\mathbb{S}^{d-1}$ denotes the unit hypersphere in $\mathbb{R}^d$ — the set of all vectors with length 1. Because all embeddings have unit norm, cosine similarity reduces to the dot product, and the **cosine distance**

Table 1: Notation summary.

Symbol	Meaning
r	A single requirement sentence (natural-language text)
$g(r) \in \mathbb{S}^{d-1}$	Unit-vector embedding of requirement r ($d = 1024$)
$d_{\cos}(u, v)$	Cosine distance between vectors u and v: $1 - u^\top v$
P_*	The *target project* under analysis (a set of requirements)
$\mathcal{D} = \{P_1, \ldots, P_M\}$	The *reference corpus*: $M = 14$ training projects (in LOO-CV)
$\mathbf{X} = \{g(r) : r \in \mathcal{D}\}$	All corpus embeddings (reference point cloud)
$\mathbf{Y}_* = \{g(r) : r \in P_*\}$	Target project embeddings
$\mathbf{Y}_j = \{g(r) : r \in P_j\}$	Embeddings of training project P_j
$K_c = 12$	Number of requirement categories (e.g., Security, Performance, Usability)
$c \in \{1, \ldots, K_c\}$	A requirement category index
$h(r) \in \{1, \ldots, K_c\}$	Hard category assignment: nearest category centroid
$\boldsymbol{\mu}_c$	Unit-vector centroid of category c in embedding space
K_s	Number of latent topics discovered by BERTopic
$\boldsymbol{\pi}(r) \in \Delta^{K_s-1}$	Soft topic distribution: $\pi_s(r)$ = probability r belongs to topic s
$\varepsilon = 10^{-8}$	Small constant to prevent division by zero

between two embeddings u and v is simply:

$$d_{\cos}(u, v) = 1 - u^\top v. \tag{2}$$

A distance of 0 means identical direction (perfect similarity); a distance of 1 means orthogonal (unrelated); a distance of 2 means opposite.

3.2 Category Centroids

We assign each requirement to one of $K_c = 12$ quality-attribute categorys (e.g., Security, Performance, Usability) using the PROMISE ground-truth labels. For each category t, the **category centroid** $\boldsymbol{\mu}_c$ is the normalised mean of all embeddings in that class:

$$\boldsymbol{\mu}_c = \frac{\sum_{r \in \mathcal{R}_c} g(r)}{\left\| \sum_{r \in \mathcal{R}_c} g(r) \right\|}, \tag{3}$$

where $\mathcal{R}_c = \{r : \text{ground-truth category of } r = c\}$ is the set of all requirements of category c. For a new requirement without a label, the **hard category assignment** assigns it to the nearest centroid:

$$h(r) = \arg\min_{c \in \{1, \ldots, K_c\}} d_{\cos}\big(g(r),\, \boldsymbol{\mu}_c\big). \tag{4}$$

This achieves 78.3% accuracy on PROMISE.

3.3 Soft Topic Distributions

BERTopic [5] discovers K_s latent topics from the corpus via UMAP [9] $\rightarrow$ HDBSCAN [8] $\rightarrow$ c-TF-IDF. Each requirement receives a **soft topic distribution**:

$$\boldsymbol{\pi}(r) = \big(\pi_1(r), \ldots, \pi_{K_s}(r)\big) \in \Delta^{K_s-1}, \tag{5}$$

where $\pi_s(r) \geq 0$ is the probability that requirement r belongs to topic s, and $\sum_s \pi_s(r) = 1$. The notation Δ^{K_s-1} denotes the probability simplex (the set of all valid probability vectors of length K_s). Unlike hard clustering, every requirement contributes to every topic with appropriate weight.

4 Method

GEOGAP computes a gap score $\Psi[c]$ for each requirement category c (e.g., Security, Performance), indicating how much the target project P_* is under-covered in that category relative to the corpus. A positive $\Psi[c]$ means the target has less coverage than typical; a negative value means surplus. The score is built from three complementary components, illustrated in Figure 2.

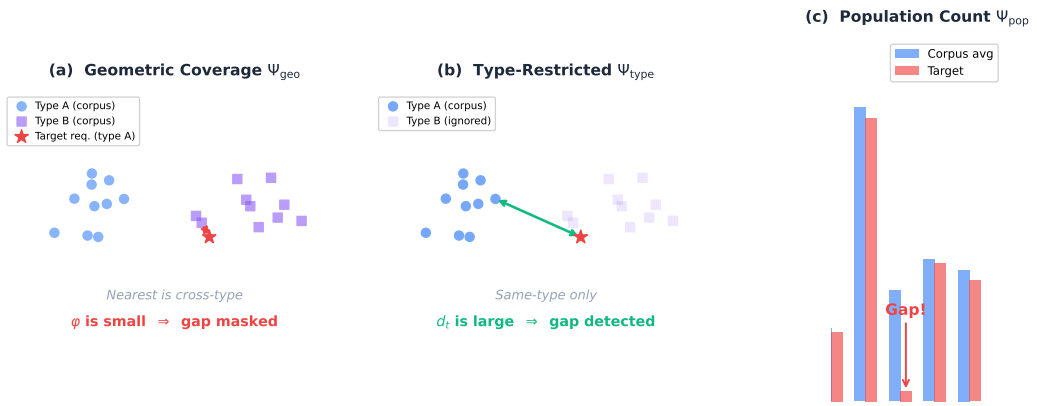


Figure 2: Three scoring components. **(a)** Geometric coverage (Ψ_{geo}): unrestricted k-NN finds a cross-category neighbour, masking the gap. **(b)** Type-restricted coverage (Ψ_{cat}): restricting to same-category neighbours reveals the gap. **(c)** Population count (Ψ_{pop}): the target has far fewer Performance requirements than the corpus average.

4.1 Geometric Coverage (Ψ_{geo})

The first component asks: for each reference requirement x in the corpus, does the target project P_* have *any* requirement nearby?

Step 1: Coverage distance. Let $x \in \mathbf{X}$ be a corpus requirement (i.e., a requirement from one of the training projects), and let $\mathbf{Y}_* = \{g(r) : r \in P_*\}$ be the set of embeddings of the target project. We find the k nearest neighbours of x in $\mathbf{Y}_*$ and compute the average distance:

$$\varphi(x;\, \mathbf{Y}_*) = \frac{1}{k} \sum_{i=1}^{k} d_{\cos}\big(x,\, y_{(i)}\big), \tag{6}$$

where $y_{(1)}, y_{(2)}, \ldots, y_{(k)}$ are the k nearest neighbours of x in $\mathbf{Y}_*$, ordered by increasing cosine distance. Intuitively, φ measures "how far is x from the closest target requirement?" A large φ means the target has nothing nearby — but this alone is insufficient, because some regions of embedding space are inherently sparse (e.g., few projects address portability). Figure 3 illustrates this distinction.

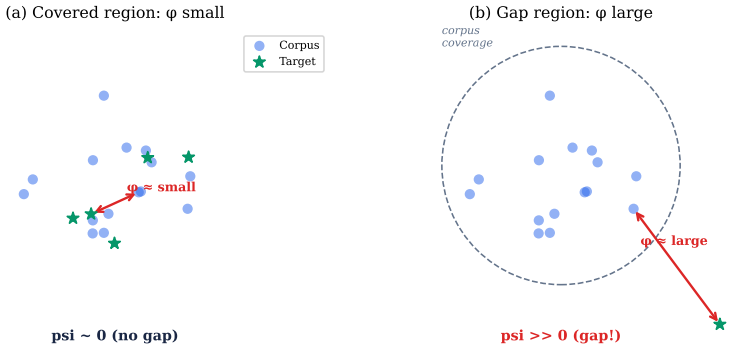


Figure 3: Geometric intuition. (a) A covered region: the target has requirements near corpus point x, so φ is small. (b) A gap region: the target's nearest requirement is far from x, so φ is large. Per-project normalisation (Figure 4) determines whether a large φ is unusual or merely reflects inherent sparsity.

Step 2: Per-project baseline. The key idea: instead of asking "is φ large in absolute terms?", ask "is it *unusually* large compared to what a typical individual project achieves at this same corpus point?" (Figure 4).

For the same corpus point x, we compute $\varphi(x; \mathbf{Y}_j)$ for each of the M training projects P_j individually. Let $\mathbf{Y}_j = \{g(r) : r \in P_j\}$ be the embeddings of training project j. The **baseline distance** and **baseline standard deviation** at point x are:

$$\varphi_0(x) = \frac{1}{M}\sum_{j=1}^{M} \varphi(x;\, \mathbf{Y}_j), \qquad \hat{\sigma}(x) = \sqrt{\frac{1}{M}\sum_{j=1}^{M}\big(\varphi(x;\, \mathbf{Y}_j) - \varphi_0(x)\big)^2}. \tag{7}$$

Here $\varphi_0(x)$ is the average coverage distance that a typical project has at point x, and $\hat{\sigma}(x)$ measures how much this varies across projects.

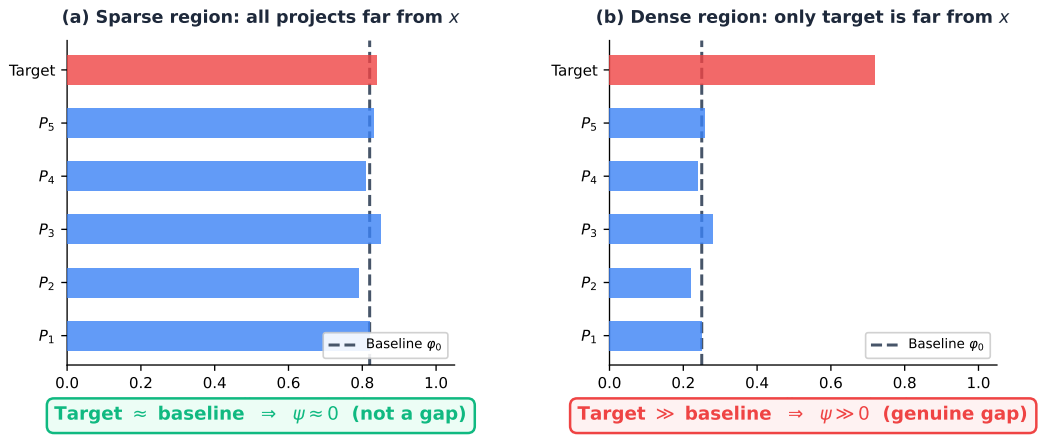


Figure 4: Per-project normalisation. (a) In a sparse region, all projects are equally far from x, so the target is typical ($\psi \approx 0$, not a gap). (b) In a dense region, training projects are close but the target is far ($\psi \gg 0$: genuine gap).

Step 3: Normalised gap score. The **per-point gap score** $\psi(x)$ is a z-score: how many standard deviations the target's distance is above the baseline:

$$\boxed{\psi(x) = \mathrm{clip}\left(\frac{\varphi(x;\, \mathbf{Y}_*) - \varphi_0(x)}{\hat{\sigma}(x) + \varepsilon},\ -5,\ 5\right)} \tag{8}$$

where $\mathrm{clip}(z, -5, 5)$ bounds the score to $[-5, +5]$ to prevent extreme outliers, and $\varepsilon = 10^{-8}$ prevents division by zero when all projects have identical distance.

Interpretation: $\psi(x) \gg 0$ means the target is unusually far from corpus point x compared to other projects (a gap signal). $\psi(x) \approx 0$ means the target is typical. $\psi(x) \ll 0$ means the target has surplus coverage at x.

Step 4: Aggregate to category level. The per-point scores $\psi(x)$ are aggregated to produce a single score per category via cell aggregation (detailed in Section 4.5):

$$\Psi_{\mathrm{geo}}[c] = \frac{\sum_s W[c,s] \cdot \Psi_{\mathrm{cell}}[c,s]}{\sum_s W[c,s]}, \tag{9}$$

where $\Psi_{\mathrm{cell}}[c,s]$ is the gap score for cell (c,s) in the (category $\times$ topic) grid, and $W[c,s]$ is the corpus weight in that cell (both defined in Section 4.5).

Why $k = 1$. On the high-dimensional hypersphere $\mathbb{S}^{d-1}$ with $d = 1024$, a phenomenon called *concentration of measure* [7] causes all neighbours beyond the 1st to lie at approximately the same "ambient" distance, regardless of the data distribution. Only the nearest neighbour carries local information. The k-sensitivity experiment (Section 5.3) confirms: GEOGAP-G AUROC decays from 0.87 (k=1) to 0.39 (k=20).

4.2 Category-Restricted Coverage (Ψ_{cat})

The geometric score Ψ_{geo} has a blind spot: when searching for the target's nearest neighbour, it considers requirements of *all* types. If the target is missing Security requirements but has nearby Performance requirements, the unrestricted k-NN will find those Performance requirements and report low distance — masking the Security gap (Figure 2a vs. b).

The category-restricted score fixes this by comparing only requirements of the *same category*.

Step 1: Type-restricted mean distance. For each category t, let $\mathbf{X}^c = \{x \in \mathbf{X} : h(x) = c\}$ be all corpus requirements assigned to category c, and let $P^c = \{y \in P : h(y) = c\}$ be all requirements in project P assigned to category c. The category-restricted distance measures how well project P covers the corpus's category-c requirements:

$$d_c(P) = \frac{1}{|\mathbf{X}^c|} \sum_{x \in \mathbf{X}^c} \min_{y \in P^c} d_{\cos}(x,\, y). \tag{10}$$

This averages, over all corpus points of category c, the distance to the project's nearest same-category requirement. If the project has no requirements of category c at all, d_t will be very large.

Step 2: Z-score against baselines. As with Ψ_{geo}, we z-score against the per-project distribution:

$$\boxed{\Psi_{\text{cat}}[c] = \text{clip}\left(\frac{d_c(P_*) - \bar{d}_c}{\hat{\sigma}_{d_c} + \varepsilon},\ -5,\ 5\right)} \tag{11}$$

where $\bar{d}_c = \frac{1}{M}\sum_{j=1}^{M} d_c(P_j)$ is the average category-restricted distance across all M training projects, and $\hat{\sigma}_{d_c} = \text{std}_j\{d_c(P_j)\}$ is its standard deviation.

Interpretation: $\Psi_{\text{cat}}[c] \gg 0$ means the target's category-c coverage is much worse than a typical project's, even when compared only against same-category requirements.

Key properties. (i) Type restriction amplifies the signal from partial removal by $\sim$1.9$\times$ vs. unrestricted k-NN (verified empirically). (ii) Because d_t averages over all $|\mathbf{X}^c|$ reference points (typically 20–190), the result is insensitive to k: AUROC is nearly flat from k=1 to k=20.

4.3 Population Count (Ψ_{pop})

The geometric scores (Ψ_{geo} and Ψ_{cat}) ask "is there *any* requirement nearby?" — a binary question. But our baseline experiments (Section 5.7) revealed that simply *counting* requirements per category with ground-truth labels achieves 0.933 AUROC. This shows that "how *many* requirements of each category?" carries strong signal that the geometric scores miss.

The population count captures this signal *without ground-truth labels* using calibrated soft assignment.

Step 1: Soft category count. Instead of hard-assigning each requirement to a single category (which has 22% error), we compute a **soft probability** that requirement y belongs to category c:

$$p_c(y) = \frac{\exp\!\big(-d_{\cos}(y, \boldsymbol{\mu}_c)\,/\,T\big)}{\sum_{c'=1}^{K_c} \exp\!\big(-d_{\cos}(y, \boldsymbol{\mu}_{c'})\,/\,T\big)}, \tag{12}$$

where $\boldsymbol{\mu}_c$ is the category centroid (Eq. 3) and $T > 0$ is a **temperature parameter** that controls the sharpness of the assignment. Small T makes the assignment nearly hard (winner-take-all); large T makes it nearly uniform. We calibrate T so that the expected confidence $\mathbb{E}_y[\max_c p_c(y)]$ matches the hard-assignment accuracy (0.783), yielding $T^* = 0.021$.

The **soft category count** for project P is the sum of these probabilities:

$$\tilde{n}_c(P) = \sum_{y \in P} p_c(y). \tag{13}$$

Intuitively, $\tilde{n}_c(P)$ is a "fractional count" of how many requirements in P are likely of category c.

Step 2: Z-score against baselines.

$$\boxed{\Psi_{\text{pop}}[c] = \text{clip}\left(\frac{\bar{\tilde{n}}_t - \tilde{n}_c(P_*)}{\hat{\sigma}_{\tilde{n}_c} + \varepsilon},\ -5,\ 5\right)} \tag{14}$$

where $\bar{\tilde{n}}_t = \frac{1}{M}\sum_{j=1}^{M} \tilde{n}_c(P_j)$ is the average soft count across training projects, and $\hat{\sigma}_{\tilde{n}_c} = \text{std}_j\{\tilde{n}_c(P_j)\}$ is its standard deviation. Note the numerator is $\bar{\tilde{n}}_t - \tilde{n}_c(P_*)$ (baseline *minus* target): a target with fewer requirements than average gets a positive score (gap).

Interpretation: $\Psi_{\text{pop}}[c] \gg 0$ means the target has substantially fewer category-c requirements than peer projects. The count signal is linear in the removal fraction ($5.3\times$ more dynamic range than the geometric score).

4.4 Fusion

The three scores capture complementary information: Ψ_{geo} is sensitive to local embedding structure but requires k=1; Ψ_{cat} provides k-robustness by averaging over all same-category reference points; Ψ_{pop} captures population-level signals that geometry alone misses. They are combined with two hyperparameters, β and γ:

$$\boxed{\Psi[c] = \underbrace{(1-\gamma)}_{\text{geometric weight}} \cdot \Big[\underbrace{\beta}_{\text{unrestricted}} \cdot \Psi_{\text{geo}}[c] + \underbrace{(1-\beta)}_{\text{category-restricted}} \cdot \Psi_{\text{cat}}[c]\Big] + \underbrace{\gamma}_{\text{count weight}} \cdot \Psi_{\text{pop}}[c]} \tag{15}$$

- $\beta \in [0, 1]$: controls the balance between unrestricted geometric coverage and category-restricted coverage. $\beta = 1$ uses only Ψ_{geo}; $\beta = 0$ uses only Ψ_{cat}.
- $\gamma \in [0, 1]$: controls how much weight the population count receives. $\gamma = 0$ ignores population; $\gamma = 1$ uses only population count.

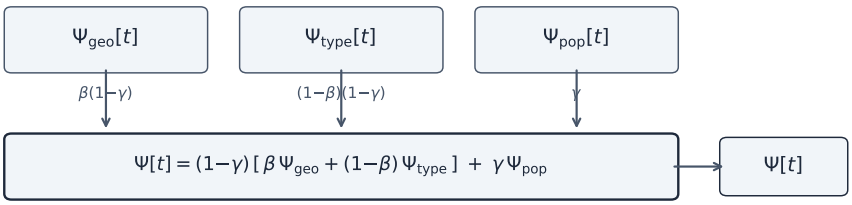


Figure 5: Score fusion via two hyperparameters. Three configurations are studied (Table 2).

Table 2: Algorithm configurations. GEOGAP (full) is recommended.

Name	Active scores	β	γ	Description
GEOGAP-G	Ψ_{geo} only	1.0	0.0	Geometric only (ablation)
GEOGAP-GC	$\Psi_{\text{geo}} + \Psi_{\text{cat}}$	0.7	0.0	Adds category-restricted (k-robust)
GEOGAP	All three	0.7	0.1	Full method

4.5 Cell Aggregation

The per-point gap scores $\psi(x)$ computed in Section 4.1 need to be aggregated from individual corpus points to the (category $\times$ topic) grid that practitioners can interpret.

Each corpus point x has a **hard category assignment** $h(x) \in \{1, \ldots, K_c\}$ (which category row it belongs to) and a **soft topic distribution** $\boldsymbol{\pi}(x) = (\pi_1(x), \ldots, \pi_{K_s}(x))$ (how its weight spreads across topic columns). The aggregated gap score for cell (c, s) — i.e., category c, topic s — is the weighted average of per-point scores:

$$\Psi_{\text{cell}}[c, s] = \frac{\sum_{x:\, h(x)=c} \pi_s(x) \cdot \psi(x)}{\sum_{x:\, h(x)=c} \pi_s(x)}. \tag{16}$$

The numerator sums the gap scores of all corpus points in category c, weighted by their topic-s membership. The denominator normalises by the total topic-s weight in category c.

The **occupancy matrix** measures how much of the target project falls in each cell:

$$\widetilde{M}_*[c, s] = \sum_{y \in P_*:\, h(y)=t} \pi_s(y), \qquad \sum_{t,s} \widetilde{M}_*[c, s] = N_*, \tag{17}$$

where N_* is the number of requirements in the target. A cell with high Ψ_{cell} and low $\widetilde{M}_*$ is a gap: the corpus expects coverage there, but the target provides little.

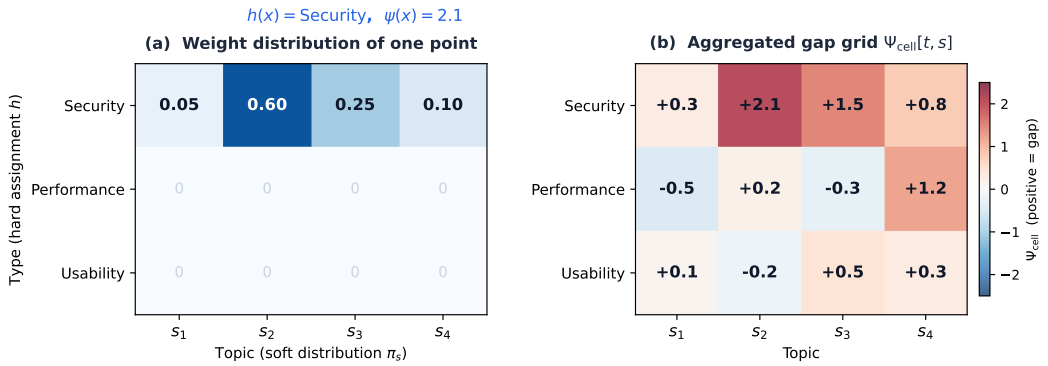


Figure 6: Soft cell aggregation. (a) A reference point distributes its score across topic columns via $\boldsymbol{\pi}$. (b) Aggregated grid: red = gap, blue = surplus.

4.6 Dual-Mode Comparison

The per-project distances $\varphi(x; \mathbf{Y}_j)$ support two comparison modes. **Mode A** (RQ1) uses uniform weights: $\varphi_0(x) = \frac{1}{M} \sum_j \varphi(x; \mathbf{Y}_j)$, detecting gaps relative to the entire corpus. **Mode B** (RQ2) uses similarity-weighted baselines: $\varphi_0(x) = \sum_j w_j \varphi(x; \mathbf{Y}_j)$ with $w_j \propto \exp(-d_{\cos}(\boldsymbol{\mu}_*, \boldsymbol{\mu}_j)/\tau)$, where $\boldsymbol{\mu}_*$ and $\boldsymbol{\mu}_j$ are project-level centroids and τ is a temperature. Mode B weights similar projects more heavily, detecting gaps relative to the target's domain peers. Optionally, project similarity can be computed via entropic optimal transport [3].

5 Experiments

We lack ground-truth gap labels, so all experiments use **synthetic gap injection**: remove requirements from well-covered categories, then check whether the detector ranks depleted categories as top gaps. All experiments use LOO-CV across 15 projects.

5.1 Type-Level Gap Detection

Question. Can the detector identify which requirement categories are completely absent from a project?

Protocol. For each LOO fold: compute $\Psi[c]$ for all 12 categories; select 1–3 best-covered types; remove all their requirements ($f = 1.0$); recompute $\Psi'[c]$; rank types by Ψ' descending; evaluate whether the depleted categories rank near the top.

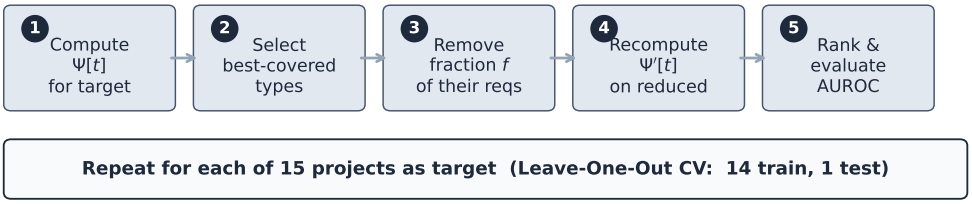


Figure 7: Evaluation protocol. Each of 15 projects takes a turn as target; well-covered categories are identified, their requirements removed, and the detector is evaluated on whether it ranks those types as gaps.

Metrics. *AUROC* (Area Under the ROC Curve) measures ranking quality: can the detector rank truly-depleted categories above non-depleted ones? AUROC $= 1.0$ means perfect separation; 0.5 means random. This is the primary metric because gap detection is fundamentally a *ranking* task—the practitioner examines top-ranked types first. *MRR* (Mean Reciprocal Rank) measures how early the first true gap appears: $\mathrm{MRR} = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{r_i}$, where r_i is the rank of the i-th true gap. A high MRR means the practitioner finds a real gap quickly.

Table 3: Category-level detection (f=1.0, k=1). GeoGap achieves 0.935 AUROC for $N \geq 50$.

	AUROC ($N\geq50$)	**AUROC** (All)	**MRR** (All)
GeoGap-G (geometric only)	0.871 ± 0.22	0.764 ± 0.26	0.453
GeoGap-GC (+ category-restricted)	0.902 ± 0.15	0.795 ± 0.21	0.422
GeoGap (full)	$\mathbf{0.935 \pm 0.14}$	$\mathbf{0.801 \pm 0.20}$	**0.472**

Finding. The full GeoGap improves over geometric-only by $+0.064$ AUROC for large projects ($N \geq 50$), with gains concentrated on GeoGap-G's weakest cases (PROMISE_8: $+0.148$; PROMISE_4: $+0.200$). The MRR of 0.472 means that, on average, the first correctly identified gap appears around rank 2.

5.2 Fraction Monotonicity

Question. Does the detector's output scale with the severity of the gap? A reliable detector should produce higher scores when *more* requirements are removed.

Protocol. Same as above, with removal fractions $f \in \{1.0, 0.75, 0.50\}$ (complete, three-quarter, and half removal).

Metric. AUROC at each fraction. We expect monotonicity: $\mathrm{AUROC}(f{=}1.0) > \mathrm{AUROC}(f{=}0.75) > \mathrm{AUROC}(f{=}0.50)$. A violation would indicate the detector responds to something other than gap magnitude.

Finding. Monotonicity is satisfied for both configurations, confirming the detector's output tracks gap severity. However, the steep drop ($0.935 \rightarrow 0.483$ for GeoGap) reveals a structural limitation: in d=1024 with small point clouds, removing 25% of a category's requirements barely shifts the k-NN geometry—surviving requirements still provide coverage for most reference points. The method is reliable for detecting *complete* absences but not partial underspecification.

5.3 k-Sensitivity

Question. How sensitive is detection to the number of nearest neighbours k? Is the gap signal strictly local (carried by the 1-NN only), and does GeoGap eliminate this sensitivity?

Protocol. Category-level injection with $f = 1.0$, varying $k \in \{1, 3, 5, 10, 20\}$.

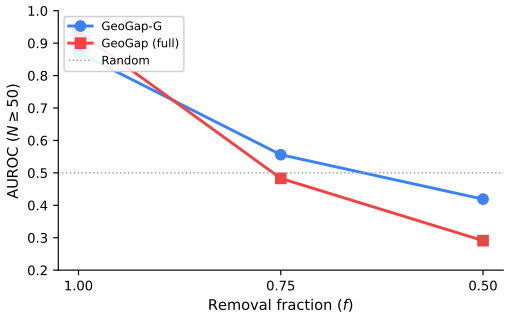


Figure 8: AUROC vs. removal fraction ($N \geq 50$). Monotonicity holds; the steep drop from f=1.0 to f=0.75 confirms the method primarily detects complete absences.

Metric. AUROC at each k. A monotonic decay with k proves the signal is local (concentration of measure on $\mathbb{S}^{d-1}$). A flat curve proves the method is k-robust. A random detector would show a constant line at $\sim$0.5 regardless of k—any deviation from flatness is evidence of a real signal.

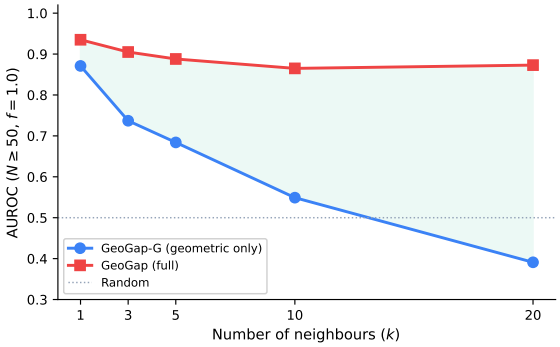


Figure 9: k-sensitivity ($N \geq 50$, f=1.0). GeoGap-G decays from 0.87 to 0.39 (strictly local signal). GeoGap stays above 0.87 at all k (robust). A random detector would be flat at 0.5.

Finding. GeoGap-G decays monotonically ($0.871 \rightarrow 0.391$), confirming the gap signal is strictly local: only the 1-nearest neighbour carries coverage information on $\mathbb{S}^{1023}$. This monotonic decay is also the *strongest evidence that the detector captures a real signal*—a noise-based detector cannot produce this pattern.

The full GeoGap is nearly flat ($0.935 \rightarrow 0.873$, a drop of only 0.062 across a $20\times$ change in k). This is the most important structural improvement: GeoGap-G has a critical hyperparameter (k=1) that must be set exactly; GeoGap does not. The robustness arises because Ψ_{cat} and Ψ_{pop} aggregate across all reference points per category *before* computing the z-score.

5.4 Permutation Test

Question. Are the observed per-project AUROCs statistically significant, or could they arise by chance given the small number of types ($K_c = 12$)?

Protocol. For each project, run 1000 random permutations: label 3 types as "targets" without removing anything, compute AUROC of the actual $\Psi[c]$ scores against these random labels. This builds a null distribution of AUROC under H_0: "scores are independent of which types are targets."

Metric. The null 95th percentile ($\approx$0.83 for K_c=12 with 3 targets). A project is *statistically significant* if its observed AUROC from the injection experiment exceeds this threshold. This tells us whether the detection result for that project could be explained by chance.

Finding. Under GEOGAP, 9 of 15 projects exceed the null 95th percentile: all 6 large projects ($N \geq 50$) and 3 small ones. The 6 non-significant projects all have $N < 50$, confirming that detection power depends on project size—small point clouds on $\mathbb{S}^{1023}$ provide insufficient geometric contrast.

5.5 Cell-Level Localisation

Question. Can the detector pinpoint gaps to specific (category $\times$ topic) cells, not just types? This tests whether the method can say "you are missing Security requirements *about encryption*" rather than just "you are missing Security requirements."

Protocol. Same injection protocol, but operating on the full cell grid ($K_c \times K_s \approx 96$ cells) with 5 target cells.

Metric. AUROC over all cells. The search space is $\sim 8\times$ larger than category-level, requiring finer geometric discrimination.

Finding. AUROC = 0.51 (indistinguishable from random). With 621 corpus points spread across ~ 96 cells (~ 6.5 per cell on average), removing requirements from a single cell barely shifts the k-NN geometry. This is an important negative result: it bounds the method's spatial resolution at the current corpus size. A corpus 5–10$\times$ larger may enable cell-level detection.

5.6 Whole-Project Holdout

Question. If an entire project is removed from the training corpus, do its dominant types now appear as gaps for similar target projects? This tests corpus-level sensitivity rather than target-level sensitivity.

Protocol. Remove P_j from training; for each other project P_k as target, check whether the categories where P_j contributed $>20\%$ of corpus mass rank as gaps.

Metric. AUROC over the 12 categories, treating dominant types of the removed project as positives.

Finding. 84 valid project pairs; mean AUROC = 0.50 (random). With $M{=}15$ projects, removing one shifts the per-project baseline $\varphi_0(x)$ by only $\sim 7\%$—well within the noise. This protocol would require a larger, more diverse corpus where individual projects have distinctive category signatures.

5.7 Baseline Comparison

Question. Does each component of the pipeline contribute measurable value, or could a simpler method achieve comparable results? Without this experiment, a reviewer could argue the entire pipeline is unnecessary.

Protocol. Six baselines are evaluated on the *identical* injection protocol ($f{=}1.0$, LOO-CV) as our method. Each ablates exactly one pipeline component (Table 4).

Metric. AUROC, compared head-to-head. A baseline that matches or exceeds GEOGAP would demonstrate the ablated component is unnecessary.

Table 4: Baselines ($f{=}1.0$, $k{=}1$). Each ablates one pipeline component.

Method	**What it tests**	**AUROC** ($N{\geq}50$)	**AUROC** (All)
Random	Null	0.345	0.309
Count (GT labels)	Is ML needed? (unfair oracle)	<u>0.933</u>	0.795
TF-IDF + k-NN	Are neural embeddings needed?	0.615	0.549
Classifier [6]	Is geometric scoring needed?	0.673	0.573
MMD/type [4]	Is normalisation needed?	0.265	0.230
Centroid dist.	Is normalisation needed?	0.284	0.266
GEOGAP-G (ours)	Geometric only	0.871	0.764
GEOGAP (ours)	Full method	**0.935**	**0.801**

Validated findings:

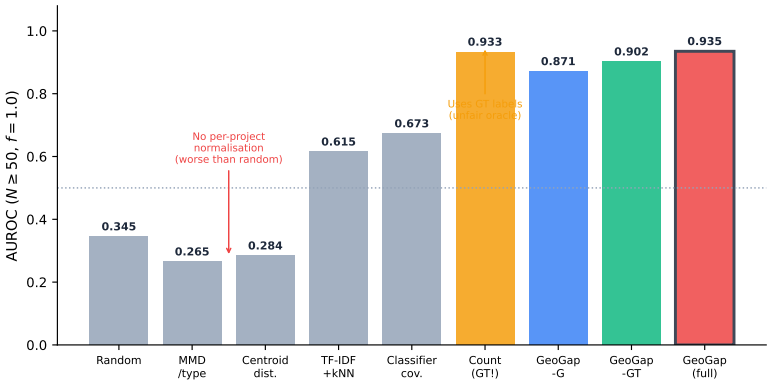


Figure 10: Baselines ($N \geq 50$, f=1.0). Methods without per-project normalisation score below random. GEOGAP matches the GT oracle without annotation.

1. **Per-project normalisation is essential.** MMD (0.265) and centroid distance (0.284) score *below* random (0.345). Without the per-project baseline $\varphi_0(x)$, raw distances cannot distinguish genuine gaps from inherently sparse regions—the very problem Section 4.1 motivates.
2. **Neural embeddings outperform bag-of-words.** GEOGAP-G (0.871) vs. TF-IDF (0.615): $+42\%$ relative improvement with the same k-NN pipeline, demonstrating that contextual embeddings capture semantic structure TF-IDF misses.
3. **Geometric scoring beats classify-and-count.** GEOGAP-G (0.871) vs. classifier coverage (0.673). The classifier approach (inspired by NoRBERT [6]) predicts types then counts; its 22% classification error propagates into the coverage estimate. Operating in continuous embedding space avoids this.
4. **GEOGAP matches the ground-truth count oracle.** GEOGAP (0.935) vs. GT counting (0.933) for $N \geq 50$—*without requiring human annotation*. The count baseline uses ground-truth category labels (an unfair advantage unavailable at deployment); the population score Ψ_{pop} captures the same signal via Gibbs soft assignment.

5.8 Summary

Table 5: Consolidated findings.

Experiment	AUROC	Verdict	Key finding
Category-level, $N \geq 50$	0.935	Strong	Matches GT oracle
Category-level, all	0.801	Good	Small projects degrade
Fraction f=0.75	0.48	Weak	Partial gaps hard
k-sensitivity	$0.94 \rightarrow 0.87$	Robust	GEOGAP-G decays to 0.39
Permutation test	—	9/15	All large projects pass
Cell-level	0.51	Random	Grid too sparse
Holdout	0.50	Random	Needs larger corpus

6 Inference

All corpus artifacts are pre-computed. For a new project: (1) embed; (2) assign topics; (3) compute all three scores; (4) aggregate; (5) report. Cost: <2 seconds. Minimum $N_* \geq 50$ for reliable detection.

7 Limitations

1. AUROC 0.94 for complete removal but $\sim$0.50 for partial gaps ($f \leq 0.75$).
2. Category assignment is 78.3% accurate; structured errors create phantom gaps.

3. 621 points across 96 cells is too sparse for cell-level localisation.
4. Gaps are corpus-relative; uncovered concerns cannot be flagged.
5. No practitioner validation of detected gaps.

8 Future Work

Several directions remain open for strengthening and extending GEOGAP.

Comparison with transformer-based gap detection. Our method treats the sentence encoder as a frozen feature extractor and operates entirely in the resulting embedding space. An alternative family of approaches trains transformer architectures end-to-end for gap- or anomaly-related objectives—for example, fine-tuning BERT-based classifiers to predict whether a requirement set is "complete" relative to a reference taxonomy, or using sequence-to-set models that directly output missing categories. A systematic comparison between our geometry-based approach and such discriminative transformer baselines would clarify when post-hoc geometric analysis suffices and when end-to-end training is necessary, particularly for the partial-gap regime ($f \leq 0.75$) where GEOGAP currently struggles.

LLM-driven gap analysis. Large language models can be prompted to analyse a requirements document and enumerate missing concerns in a single forward pass (e.g., "*Given these requirements, what quality attributes are underspecified?*"). Such zero-shot or few-shot LLM approaches require no corpus construction and can leverage world knowledge beyond the training data. However, they lack the quantitative, per-category scoring that GEOGAP provides, and their outputs are difficult to calibrate or reproduce. A comparative study measuring precision, recall, and inter-run consistency of LLM-driven gap analysis against GEOGAP would establish the relative strengths of each paradigm—and whether combining them (e.g., using GEOGAP scores as structured context for an LLM's analysis) yields further improvement.

Human expert validation. All current experiments evaluate the detector against its own synthetic gap definition. The most important missing piece is a *practitioner study*: present GEOGAP reports to requirements engineers working on real projects, measure whether flagged gaps are judged actionable, and compare against expert-identified gaps that the detector misses. Such a study would move evaluation from "can the detector recover artificially removed types?" to "does the detector help engineers write better specifications?"—which is the ultimate measure of practical value.

Domain-adapted embeddings via fine-tuning. GEOGAP currently uses a general-purpose sentence encoder (Qwen3-Embedding-0.6B) without any task-specific adaptation. The embedding space therefore reflects general semantic similarity rather than requirement-coverage structure: two requirements may be semantically close but address different quality attributes, or semantically distant but both relevant to the same gap. Fine-tuning the encoder with a contrastive objective designed for gap detection—for instance, pulling same-category requirements closer and pushing cross-category requirements apart, or training with triplets derived from the synthetic injection protocol itself—could reshape the embedding space to amplify the geometric signal that GEOGAP relies on. This is especially promising for the partial-gap regime, where the current frozen embeddings provide insufficient geometric contrast between "75% covered" and "fully covered" types.

References

[1] Alibaba Cloud. Qwen3-embedding. `https://huggingface.co/Qwen/Qwen3-Embedding-0.6B`, 2025.

[2] Jane Cleland-Huang, Raffaella Settimi, Xuchang Zou, and Peter Solc. The detection and classification of non-functional requirements with application to early aspects. In *14th IEEE International Requirements Engineering Conference (RE'06)*, pages 39–48. IEEE, 2006. doi: 10.1109/RE.2006.65.

[3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems 26 (NeurIPS)*, pages 2292–2300, 2013.

[4] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773, 2012.

[5] Maarten Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794*, 2022.

[6] Tobias Hey, Jan Keim, Anne Koziolek, and Walter F. Tichy. NoRBERT: Transfer learning for requirements classification. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*, pages 169–179. IEEE, 2020. doi: 10.1109/RE48521.2020.00028.

[7] Michel Ledoux. *The Concentration of Measure Phenomenon*, volume 89 of *Mathematical Surveys and Monographs*. American Mathematical Society, 2001.

[8] Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017. doi: 10.21105/joss.00205.

[9] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

A Dataset Selection: PROMISE NFR

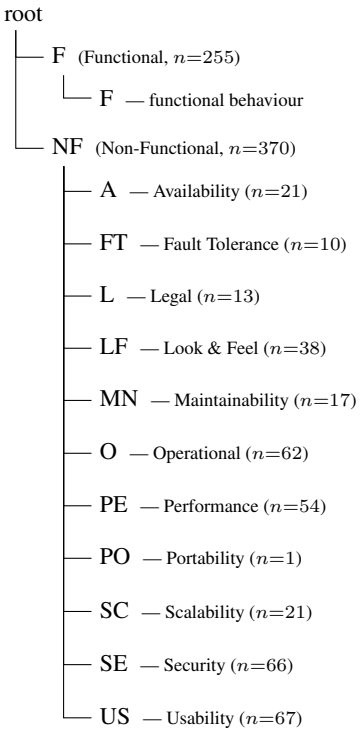
The PROMISE NFR dataset [2] is a widely used benchmark for software requirement classification containing $N = 625$ requirement sentences from 15 projects, each annotated with a hierarchical category label.

A.1 Data Format

Each record r_i is a tuple $r_i = (\texttt{text}_i, y_i^{(1)}, y_i^{(2)}, \texttt{pid}_i)$, $i = 1, \ldots, 625$, where $\texttt{text}_i$ is a natural-language requirement sentence, $y_i^{(1)} \in \{\text{F}, \text{NF}\}$ is the Level-1 label (Functional vs. Non-Functional), $y_i^{(2)} \in \mathcal{Y}_2$ is the Level-2 sub-class ($|\mathcal{Y}_2| = 12$), and $\texttt{pid}_i \in \{1, \ldots, 15\}$ is the source project ID. Every L2 label maps deterministically to an L1 label via a parent function π: $y_i^{(1)} = \pi(y_i^{(2)})$.

A.2 Label Taxonomy

The labels form a two-level tree. Level 1 splits requirements into Functional (F) vs. Non-Functional (NF). Level 2 provides 12 sub-classes:



A.3 Class Distribution

The dataset is imbalanced at both levels. The largest class (F, n=255) is $255\times$ larger than the smallest (PO, n=1). At L1, the split is NF : F $= 370 : 255 \approx 59 : 41$. Five classes have $n_k < 25$, making per-class metrics noisy. These imbalances motivate our use of Macro-F1 (which weights all classes equally) rather than accuracy as the primary classification metric.

A.4 How We Use This Dataset

We use PROMISE NFR in two stages: **(1) embedding model selection**—evaluating candidate encoders via zero-shot nearest-prototype classification directly on this dataset; and **(2) gap detection experiments**—using the selected model (Qwen3-Embedding-0.6B) and the 12-class taxonomy as the category axis of our coverage analysis. The model selection protocol requires no training data; it tests how well the embedding space separates categories from label semantics alone.

B Embedding Model Selection

B.1 Motivation

Our gap detection pipeline operates on sentence embeddings $\mathbf{x}_i = E(r_i) \in \mathbb{R}^d$. The quality of every downstream result depends on the encoder E. General-purpose benchmarks (MTEB) rank models across diverse tasks, but a high MTEB score does not guarantee strong performance on short software-requirement sentences with a hierarchical taxonomy. We therefore conducted a **task-specific model selection** on the PROMISE NFR dataset.

B.2 Selection Metrics

Primary: Nearest-Centroid Macro-F1 (zero-shot). For each candidate model and description variant $v \in \{\texttt{name_only}, \texttt{short}, \texttt{detailed}\}$: embed all N requirements, embed the 12 L2 label descriptions as centroids, classify by nearest centroid, report Macro-F1 for the best variant.

Supporting metrics. *H-Score*: hierarchy-aware accuracy using normalised tree-path distance. *ARI/NMI/V-Measure*: K-Means clustering alignment with ground-truth labels. *Linear Probe F1*: supervised logistic regression on frozen embeddings (5-fold CV) as an upper bound.

B.3 Results

Table 6: Model selection on PROMISE NFR. Ranked by NC Macro-F1 (L2).

Model	NC F1 (L2)	H-Score	ARI	LP F1
Qwen3-Embedding-0.6B	**.442**	.745	**.157**	.715±.024
Qwen3-Embedding-4B	.435	.717	.088	.758±.028
qwen3-embedding:8b	.427	**.790**	.106	**.763**±.040
stella_en_400M_v5	.402	.617	.095	.757±.036
gte-large-en-v1.5	.400	.711	.126	.704±.061
nomic-embed-text-v1.5	.389	.643	.077	.729±.068
e5-large-v2	.373	.660	.069	.744±.038
bge-large-en-v1.5	.353	.709	.141	.723±.037
all-MiniLM-L6-v2	.296	.578	.078	.647±.044

Key findings. (1) Qwen3-Embedding-0.6B leads on the primary metric (NC F1 = 0.442) and clustering alignment (ARI = 0.157). (2) Within the Qwen family, 0.6B $>$ 4B $>$ 8B on NC-F1, while 8B is strongest on LP-F1/H-Score; model size helps some space properties but not all. (3) The `short` description variant is best for 10 of 11 models. (4) The supervised–zero-shot gap remains large (0.442 vs. 0.763), confirming label-description matching captures only part of the class structure.

Selection. For downstream experiments requiring zero-shot nearest-centroid scoring, we select **Qwen3-Embedding-0.6B** ($d = 1024$).

C Preliminary Evaluation of the Embedding Space

Before designing the gap detection algorithm, we conducted three experiments to characterise the embedding space and validate that it provides a suitable geometric foundation.

C.1 Experiment A: Semantic Alignment (Zero-Shot Classification)

Question. Does the embedding space organise requirements in a way that aligns with human-defined categories?

Protocol. *Top-down (Test a):* Nearest-centroid classification using label description embeddings as centroids. Tested at both L1 (F/NF) and L2 (12 sub-classes) with three description variants. *Bottom-up (Test b):* K-Means, HDBSCAN, and Agglomerative Hierarchical Clustering on raw embeddings, compared to ground-truth labels via ARI, NMI, V-Measure, and Purity.

Table 7: Semantic alignment: nearest-centroid classification.

Level	Best variant	Accuracy [95% CI]	Macro-F1	κ	H-Score
L1	detailed	0.502 [0.464, 0.542]	0.481	0.102	—
L2	short	0.467 [0.429, 0.507]	0.442	0.393	0.745

Findings. (1) L2 accuracy (0.467) is nearly $6\times$ the 12-class chance baseline (0.083), with $\kappa = 0.393$ ("fair" agreement). H-Score = 0.745 confirms most errors are within the same taxonomy branch. (2) K-Means at L2 achieves ARI = 0.157 ($p < 0.001$), substantially above random, confirming the embedding space has natural clusters that align with the label taxonomy. (3) Embedding-based clustering outperforms TF-IDF by 67% (NMI: 0.323 vs. 0.193). (4) HDBSCAN fails (ARI = -0.041), consistent with concentration of measure in $d = 1024$.

Table 8: Clustering alignment at L2 ($K_2 = 12$). p-values from permutation test ($n = 1000$), BH-corrected.

Algorithm	ARI	NMI	V-Measure	Purity	p(ARI)
K-Means	**0.157**	**0.323**	**0.323**	0.594	<0.001
AHC (Ward)	0.119	0.292	0.292	0.558	<0.001
HDBSCAN	−0.041	0.038	0.038	0.408	1.000
TF-IDF + K-Means	−0.011	0.193	0.193	0.454	—
Random	0.001	0.053	0.053	0.408	—

Motivation for GEOGAP. These results show the embedding space encodes meaningful requirement-category structure, but imperfectly (78.3% centroid accuracy, ARI = 0.157). A naive classifier applied to this space would propagate the 22% error rate. This motivates the geometric k-NN approach: instead of classifying and counting, GEOGAP operates directly in the continuous embedding space, avoiding hard classification errors entirely.

C.2 Experiment B: Coverage Analysis

Question. Can the embedding space quantify how well different requirement sources cover each other?

Protocol. Embed three text sets—raw documentation, ground-truth (GT) requirements, and LLM-extracted requirements—and compare their distributions using Wasserstein-1, Sliced Wasserstein, MMD^2, and point-level coverage metrics (ANND, AUCC).

Findings (synthetic data, $N \leq 16$). (1) LLM outputs are distributionally closest to GT (lowest W_1, MMD^2, ANND). (2) 46.7% of GT requirements are "hidden" (far from any raw documentation chunk); the LLM recovers 85.7% of these. (3) The embedding geometry provides a natural framework for measuring coverage—the per-point nearest-neighbour distances that GEOGAP uses for gap detection originate from this observation.

C.3 Experiment C: Trajectory Analysis

Question. Does the embedding model's classification converge as more of a requirement's text is revealed?

Protocol. Inspired by the Landscape of Thoughts framework, embed progressively longer prefixes of each requirement and track the Gibbs-distribution probability vector over 12 category centroids at each step.

Findings ($n = 100$ requirements). (1) Mean uncertainty is 2.483, within 0.08% of maximum entropy ($\log 12 = 2.485$), indicating the Gibbs distribution is essentially uniform at $T = 1.0$. (2) Correct trajectories converge $6\times$ faster (speed = 0.166 vs. 1.000) and are 53% more consistent (0.874 vs. 0.571) than incorrect ones. (3) The near-uniform uncertainty motivates our calibrated temperature $T^* = 0.021$ for the population count score Ψ_{pop}: at $T = 1.0$, Gibbs assignment is too flat to be useful; at $T^* = 0.021$, it matches the 78.3% hard-assignment accuracy.

C.4 Cross-Experiment Synthesis

These preliminary experiments establish three facts that motivate GEOGAP:

1. The embedding space encodes requirement-category structure (L2 Macro-F1 = 0.442, ARI = 0.157), but imperfectly—motivating geometric coverage analysis over hard classification.
2. Nearest-neighbour distances in this space naturally capture coverage relationships—motivating the k-NN formulation.
3. Gibbs soft assignment at calibrated temperature provides annotation-free category counting—motivating the population score Ψ_{pop}.